

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 4

The xv6 Shell from the Outside

Done by: st. gr. FAF-241, Coșneanu Corina

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

INTRODUCTION AND OBJECTIVES **1**

1 Part 1: Fix sleep for the current xv6 **2**

2 Part 2: bash versus the xv6 shell **3**

3 Part 3: Who expands *? A program that prints its argv **6**

4 Part 4: Read the shell **9**

CONCLUSIONS **11**

INTRODUCTION AND OBJECTIVES

Objective

The primary goal of this laboratory is to analyze the inner workings of the xv6 command-line interpreter (`user/sh.c`) by contrasting it directly with the fully-featured Linux Bash shell. Through side-by-side practical experiments, the work aims to determine which functionalities (such as wildcard expansion and quote parsing) are handled intrinsically by the shell, and which rely on the individual compiled programs.

1 Part 1: Fix sleep for the current xv6

Recent updates to the xv6 operating system (specifically post-August 2025) replaced the classic `sleep()` system call with `pause()`. To ensure the `sleep` utility developed in earlier labs continues to function, the source code in `user/sleep.c` was modified to utilize the new `pause()` kernel interface.

```
vboxuser@lab1:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

Figure 1.1: The grep result in user.h showing pause declaration

```
vboxuser@lab1:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
    // Check if the user provided the exact number of arguments
    if(argc != 2) {
        fprintf(2, "Error: valid usage is sleep <ticks>\n");
        exit(1);
    }

    // Parse the input and pause execution
    int wait_ticks = atoi(argv[1]);
    pause(wait_ticks); // Replaced sleep() with pause() for the new xv6 version

    exit(0);
}
```

Figure 1.2: The updated sleep.c implementation

```
$ sleep 20
$ sleep
Error: valid usage is sleep <ticks>
```

Figure 1.3: sleep running in xv6, with and without an argument

2 Part 2: bash versus the xv6 shell

The following table documents the comparative behavior of identical command strings executed within the Linux Bash environment versus the minimalistic xv6 shell.

Command	Tests	bash	xv6
`echo hello > f; cat <f`	redirection	Outputs "hello"	Outputs "hello"
`ls | grep READ`	pipe	Prints "README"	Prints "README"
`(echo a; echo b) | wc`	grouping	Outputs line/word/byte counts	Syntax error (unsupported grouping)
`echo one; echo two`	sequence	Prints "one" then "two"	Prints "one" then "two"
`echo bg &`	background	Executes in background, prints PID	Executes in background
`ls *`	globbing	Expands to file names in directory	Prints literal "*"
`echo $HOME`	variables	Prints the path (e.g. /home/user)	Prints literal "$HOME"
`echo "a b"`	quotes	Prints a b (without quotes)	Prints "a b" (with quotes)
`mkdir d; cd d; ls ..`	PATH	Executes ls on the parent dir	ls fails (command not found)
`arrow up, Tab`	history	Shows previous command / autocompletes	Prints raw ANSI escape characters

Table 1: Command execution comparison

Analytical Observations:

1. **Common vs. Exclusive Features:** Both environments successfully support I/O redirection (`<`, `>`), pipeline chaining (`|`), sequential execution (`;`), and background processing (`&`). However, Bash exclusively supports advanced features such as wildcard globbing (`*`), variable substitution (`$`), quote stripping, command history, and environment variables (like `PATH`).

2. **The `ls` Failure in xv6:** In xv6, typing `ls ..` while inside directory `d` fails because the system lacks a `PATH` environment variable. It searches for the `ls` executable inside the current working directory. To fix this, we must provide the absolute or relative path to the root binaries, such as executing `../ls ..` or `/ls ...`

3. **Quality of Life:** The absence of command history (Arrow Up) and Tab-autocompletion in xv6 is severely noticeable during daily operations. Implementing command history would likely be the easiest feature to add by maintaining a simple string buffer in the shell's memory loop.

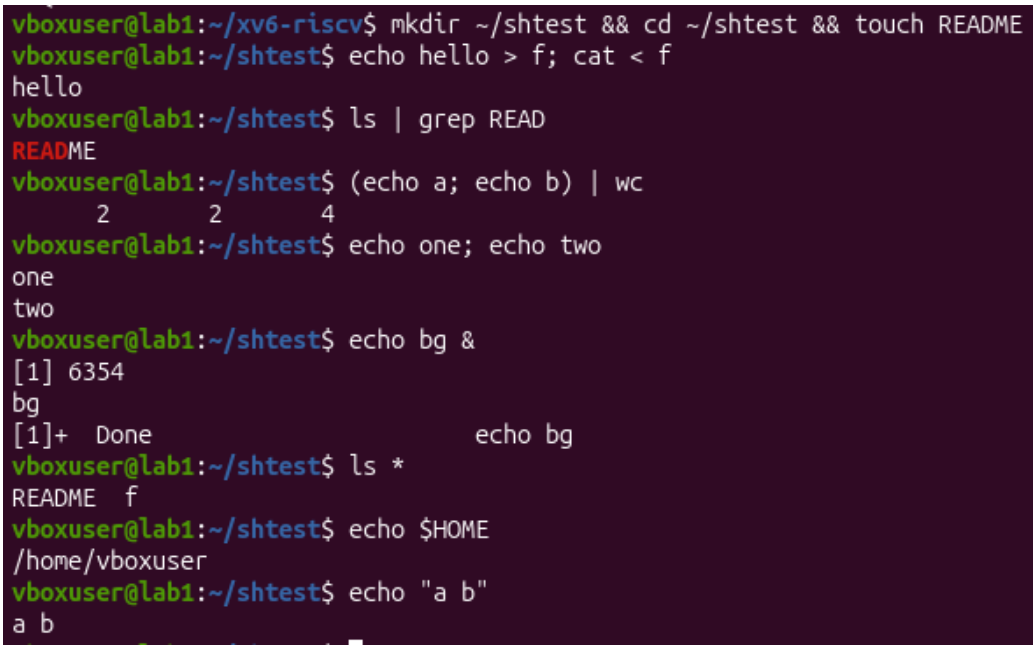


Figure 2.1: The commands run in bash

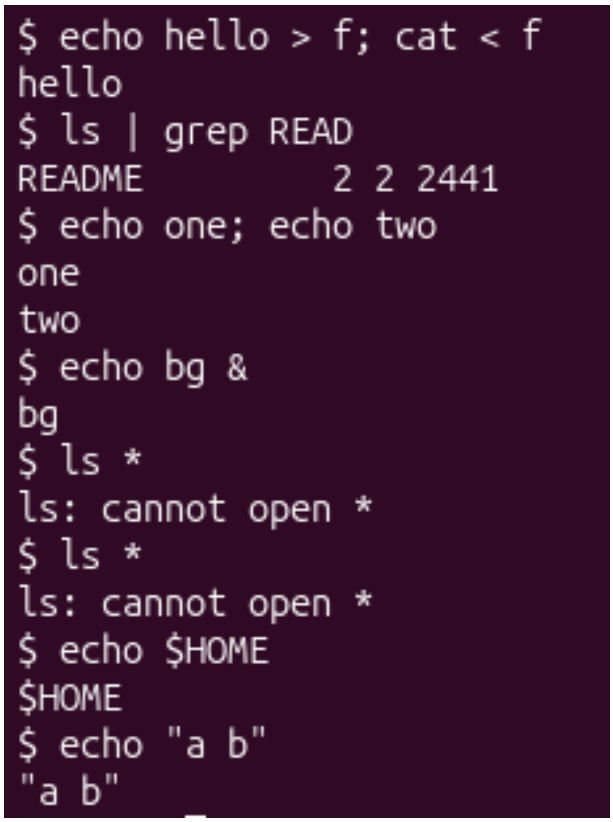


Figure 2.2: The same commands run in xv6



Figure 2.3: ls failing after cd d, and the working alternative

3 Part 3: Who expands *? A program that prints its argv

To investigate how the operating system handles string parsing before execution, a C program (`args.c`) was written to print its received argument vector (`argv`).

Analytical Observations:

1. **Argument Count:** When executing `args *.c "x y"`, Bash expanded the wildcard into individual files and stripped the quotes, resulting in multiple distinct arguments. Conversely, xv6 treated the entire string literally, passing exactly 2 arguments to the program: `*.c` and `"x y"`.

2. **Responsibility for Expansion:** This proves unequivocally that in a Unix-like environment, it is the *shell* (e.g., Bash) that handles wildcard globbing and quote removal before passing the formatted arguments to the kernel's `exec` system call. The compiled program itself plays no role in this process.

3. **Handling Spaces:** When running `args one two three`, both shells stripped the extra whitespace. The shell tokenizes the input string using spaces as delimiters, meaning any contiguous sequence of spaces is simply treated as a single boundary between arguments.

```
vboxuser@lab1:~/xv6-riscv$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
    printf("Total arguments received: %d\n", argc);

    // Loop through and print each argument
    for(int idx = 0; idx < argc; idx++) {
        printf("arg[%d] -> %s\n", idx, argv[idx]);
    }

    exit(0);
}
vboxuser@lab1:~/xv6-riscv$ grep "_args" Makefile
        $U/_args
vboxuser@lab1:~/xv6-riscv$
```

Figure 3.1: args.c and the UPROGS line in Makefile

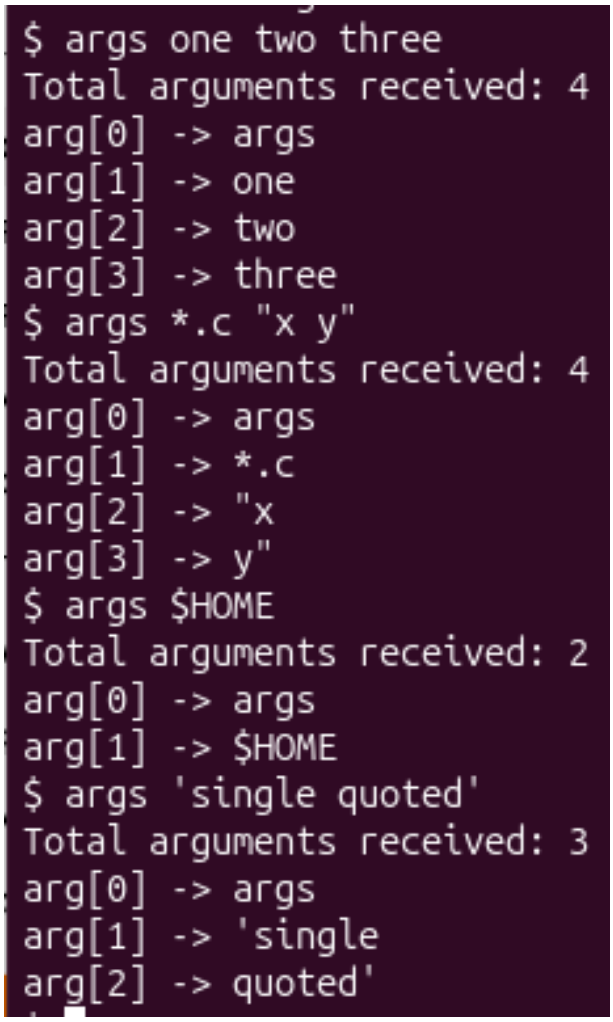


Figure 3.2: args output in xv6

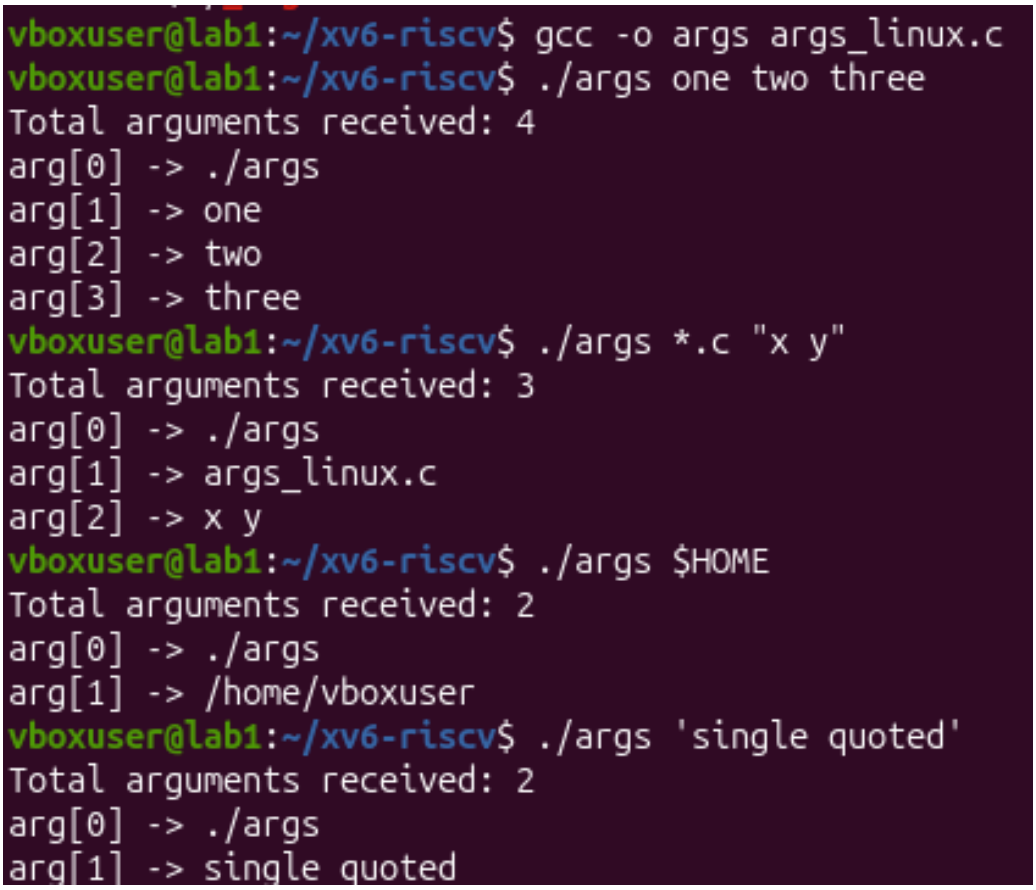


Figure 3.3: args_linux output in bash

4 Part 4: Read the shell

By analyzing the source code of the xv6 shell (`user/sh.c`), we can map its logical execution flow.

Analytical Observations:

1. **Command Types:** The shell defines five primary execution structures: `EXEC` (standard execution), `REDIR` (redirection via `<` or `>`), `PIPE` (communication via `|`), `LIST` (sequence via `;`), and `BACK` (background via `&`).

2. **The `cd` Command:** Directory navigation is handled directly in the `main` loop of `sh.c` as a built-in function utilizing the `chdir()` system call. It cannot be an external program because a child process is physically incapable of altering the working directory of its parent shell process.

3. **Absence of Advanced Parsers:** Searching the `sh.c` code for wildcard handlers (`*`) or variable parsers (`$`) yields zero results. This definitively explains the behavior observed in Part 2: the minimalist xv6 shell simply lacks the programmatic logic required to expand variables or perform globbing.

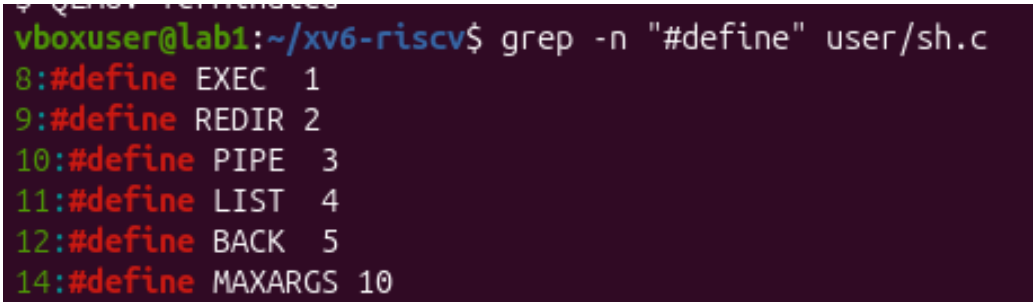


Figure 4.1: The command kinds defined in sh.c

```
vboxuser@lab1:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2: Where each kind is run

```
vboxuser@lab1:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Figure 4.3: The code that handles cd

CONCLUSIONS

This laboratory work successfully demystified the boundary between shell responsibilities and executable program logic. By comparing the minimalist xv6 shell to Linux Bash, it became evident that features often taken for granted—such as wildcard expansion, quote stripping, variable interpolation, and the `PATH` search mechanism—are entirely implemented at the shell interpreter level, not within the kernel or the compiled applications. Furthermore, reading the underlying C code of the xv6 shell reinforced theoretical operating system concepts, particularly why state-altering commands like `cd` must remain embedded as shell built-ins due to process isolation rules.